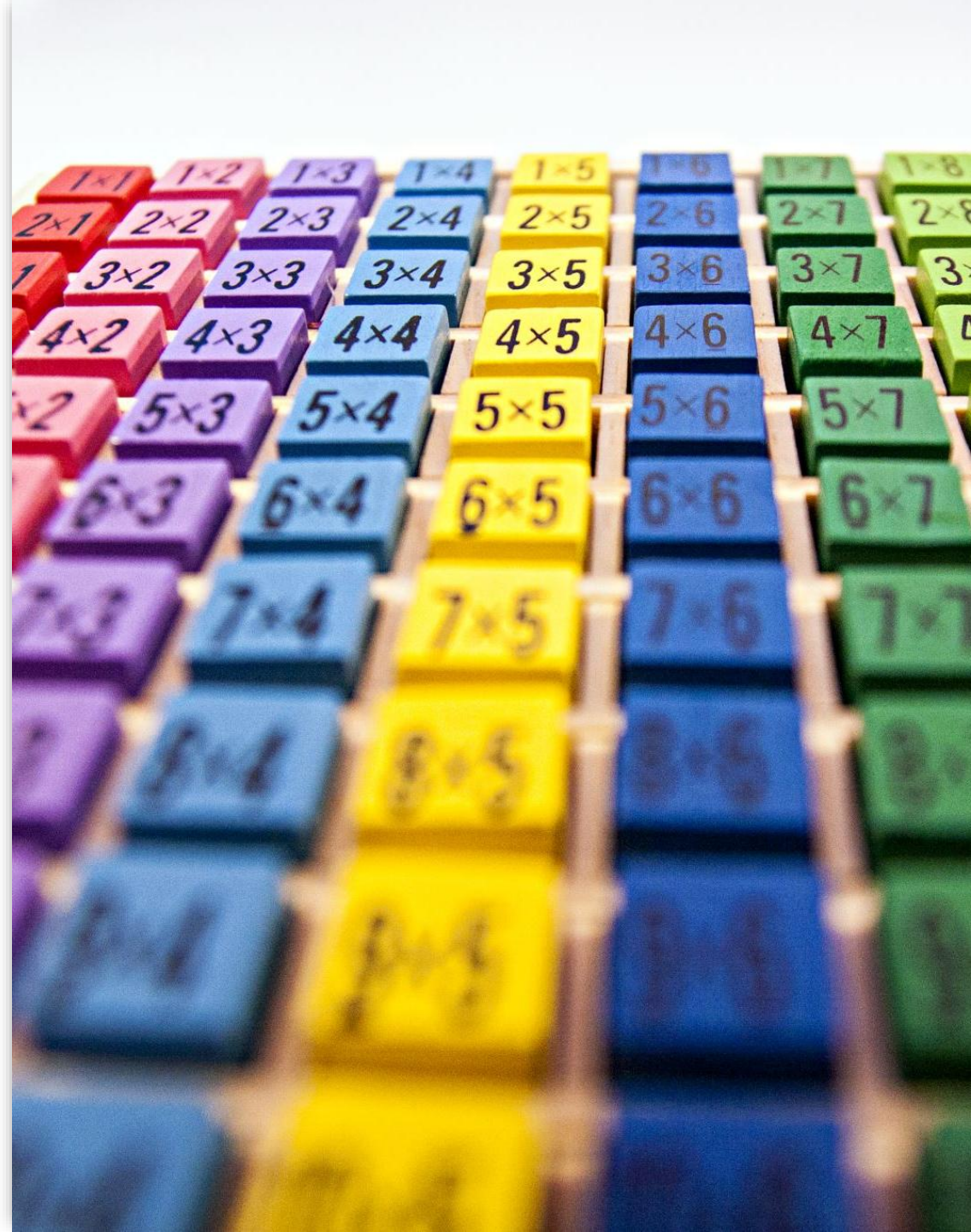


Numerical Precision Considerations

- **Challenge:** Gradients often smaller than activations
- **Quantization Errors:** Accumulate through chain rule
- **Mixed-Precision Strategy:**
 - Forward: 16-bit activations
 - Backward: 32-bit gradients
- **Benefits:** Reduced memory, maintained stability



Communication Optimization

BF16 is a 16-bit floating-point format that sacrifices precision for a wide dynamic range, making it stable for training neural networks. FP16 is a 16-bit format that offers higher precision than BF16 but has a narrower range, which can cause overflow or underflow issues during training.

ResNet-50 is a popular model for image recognition whose key innovation was a "shortcut" that allows the core signal, or meaningful information, to skip over layers. These shortcuts prevent the signal from getting weak and lost in deep networks, providing a direct path that makes them much easier to train accurately.

How big are the gradients?

- Gradient size = $P \text{ params} \times b \text{ bytes/param}$
- FP32 = 4 bytes; BF16/FP16 = 2 bytes

Ring all-reduce traffic per device:

- $2 \times P \times b \times (N-1)/N$

Example (ResNet-50 $\approx$ 25M params)

- FP32 gradients $\approx$ 100 MB
- $N = 8$ GPUs $\rightarrow \approx$ 175 MB traffic per device per step

Fabrics matter

- NVLink $\sim$ 300 GB/s $\rightarrow \sim$ 0.6 ms for 175 MB (idealized)
- 100 Gbps Ethernet/InfiniBand $\approx$ 12.5 GB/s $\rightarrow \sim$ 14 ms (idealized)

Make it faster

- Overlap comm with compute via gradient bucketing
- Choose bucket sizes to avoid many tiny messages
- Use BF16/FP16 gradients to halve traffic (with proper scaling)
- For small tensors, tree-based reductions can beat ring due to lower latency

Numerical Precision in AI Hardware

Trading Accuracy for Efficiency

Precision Format Evolution:

```
// Training: High precision for gradient stability
float32 weights[1024][512]; // 32-bit floating point
float32 activations[batch][1024];

// Inference: Reduced precision for efficiency
int8_t quantized_weights[1024][512]; // 8-bit integer
int8_t quantized_activations[batch][1024];

// Ultra-low power: Extreme quantization
int4_t binary_weights[1024][512]; // 4-bit or binary
```

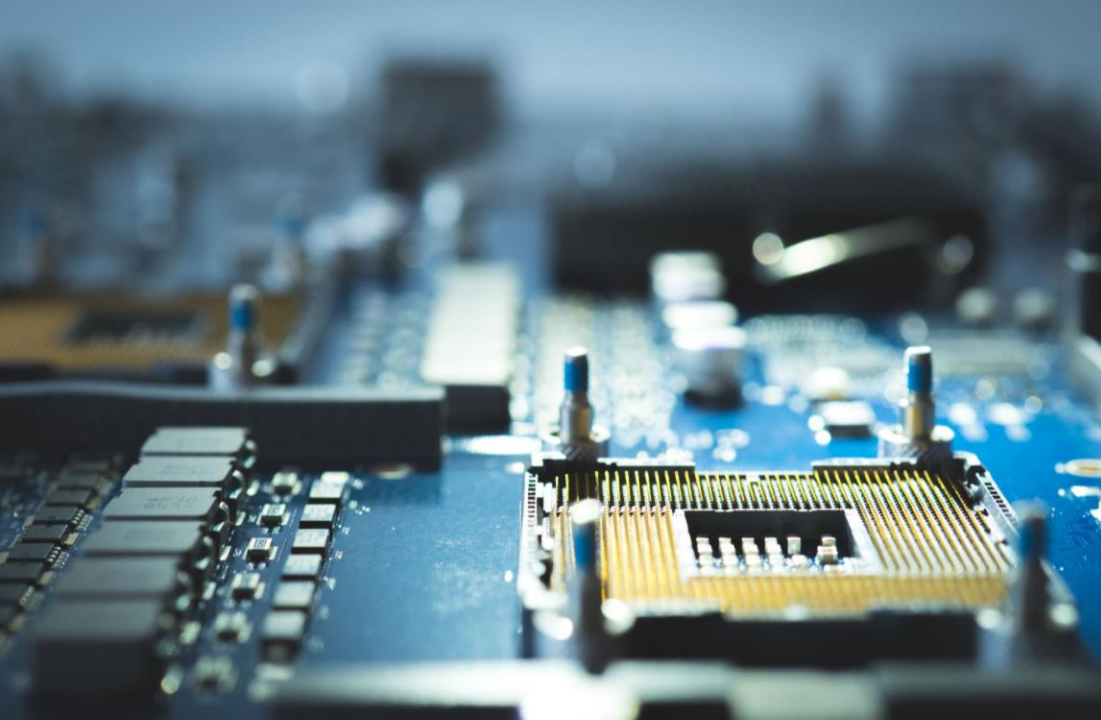
Hardware Implications:

- **FP32**: Full accuracy, high energy cost, large storage
- **FP16/BF16**: 2x memory reduction, faster computation on modern hardware
- **INT8**: 4x memory reduction, specialized integer units, minimal accuracy loss
- **INT4/Binary**: Extreme efficiency, requires careful algorithm design

Mixed-Precision Computing:

Modern accelerators support different precisions for different operations:

- Matrix multiplication: INT8 or FP16
- Accumulation: FP32 for numerical stability
- Activation functions: INT8 for efficiency



Workflow

- Check supported precisions on the target.
- Start with INT8 (mobile), FP16/INT8 (server), tighter bits only if necessary (MCU).
- Calibrate, test on device, iterate.

Quantization Strategy Selection

Goal: Pick the lowest precision your hardware supports without breaking accuracy.

Server GPUs

- Train: FP32 or mixed FP16.
- Infer: FP16/TF32/INT8 on tensor cores.
- Use higher precision if accuracy is critical.

Mobile Processors

- Default to INT8 for weights and activations.
- Use per-channel weights; keep 32-bit accumulators.
- Verify kernels run on the device accelerator, not CPU.

Microcontrollers

- Go lower if needed: INT8 → 4-bit weights where supported.
- Consider ultra-low-bit activations only when memory forces it.
- Ensure weights fit in flash and peak activations in SRAM.
- Quantization-Aware Training
- Use QAT for aggressive quantization or MCU targets to recover accuracy.